

如何防止配置文件敏感信息泄漏

在日常工作中，我们经常需要使用到各种敏感配置，如数据库密码、云厂商的 SecretId、SecretKey、支付签名等敏感信息。

通常情况下，我们会将这些敏感信息明文放到配置文件中，或者放到像 Nacos、Apollo 这样的配置中心中。然而，这种做法存在严重的安全隐患。

接下来，我们将详细讨论将敏感信息明文放到配置文件中的危害，以及如何解决这个问题。

敏感配置明文存储

在 SpringBoot 项目中，将敏感信息（如数据库密码、API 密钥等）放到配置文件中存在以下危害：

1. 安全风险：配置文件是明文存储的，如果敏感信息被存储在明文的配置文件中，那么攻击者可以通过访问配置文件来获取这些敏感信息。这种安全漏洞可能会导致数据泄露、系统受到攻击等风险。
2. 隐私泄露：敏感信息一旦泄露，可能会导致用户隐私泄露，影响用户体验和信任。例如，数据库密码泄露可能会导致攻击者获得对数据库的访问权限，进而获取用户的个人信息。
3. 维护成本高：在多个环境中使用相同的配置文件，需要在每个环境中修改敏感信息，这会增加维护成本，并且容易出现人为错误。这些错误可能导致敏感信息被错误地泄露或访问。
4. 不易追踪：如果敏感信息泄露，很难追踪泄露的原因和范围。因此，必须采取预防措施，以避免敏感信息泄露。
5. 违反安全法规：某些行业或地区的安全法规，如 PCI DSS、HIPAA 等，禁止将敏感信息明文存储在配置文件中。如果违反了这些法规，可能会面临法律风险和罚款等后果。

综上所述，将敏感信息放到配置文件中存在安全风险和隐私泄露的风险，并且增加了维护成本，并不推荐生产中如此配置。

如何配置敏感信息（理论）

为了保护敏感信息，应该采取相应的安全措施，如使用加密配置文件、使用环境变量替代明文配置信息等。

以下是几个安全措施，可以帮助保护配置文件中的敏感信息：

1. 使用环境变量替代明文配置信息。可以使用环境变量来替代配置文件中的明文敏感信息。在部署应用程序时，可以通过设置环境变量来传递敏感信息，而不是将它们明文写在配置文件中。
2. 加密配置文件。可以使用对称加密或非对称加密来加密配置文件，以保护敏感信息。在加密配置文件时，必须指定加密算法和密钥，以及解密时使用的密钥。
3. 禁止将配置文件提交到代码仓库。为了防止配置文件泄露，应该禁止将配置文件提交到代码仓库中。可以将配置文件添加到 Gitignore 文件中，防止将其提交到代码仓库中。
4. 使用安全的存储方式。如果必须将敏感信息存储在配置文件中，应该确保使用安全的存储方式。可以将配置文件存储在受保护的目录中，并限制访问该目录的权限。另外，可以使用加密文件系统来存储配置文件，以提高安全性。
5. 限制访问配置文件的权限。应该限制可以访问配置文件的用户和进程的权限，以防止未经授权的用户和进程访问配置文件。可以使用操作系统的权限控制功能，例如 Linux 中的文件权限和 SELinux 等机制，来限制访问配置文件的权限。
6. 审计配置文件的访问记录。应该定期审计配置文件的访问记录，以检测是否存在未经授权的访问。可以使用日志记录工具来记录配置文件的访问记录，并定期检查日志以发现异常行为。

为了保护配置文件中的敏感信息，可以采取多种安全措施，如使用环境变量替代明文配置信息、加密配置文件、禁止将配置文件提交到代码仓库、使用安全的存储方式、限制访问配置文件的权限以及审计配置文件的访问记录。

如何配置敏感信息（实战）

1. 环境变量替换

使用环境变量替代明文配置信息可以有效地保护敏感信息，以下是具体步骤：

1. 在配置文件中，将需要替换的敏感信息部分替换为占位符，例如将数据库密码替换为 `${DB_PASSWORD}`。
2. 在操作系统中设置环境变量，以便在运行应用程序时，可以通过环境变量传递敏感信息。例如，在 Linux 中，可以使用 `export` 命令设置环境变量，如 `export DB_PASSWORD=your_password`。
3. 在应用程序中，使用 Spring 的 `@Value` 注解来读取环境变量的值。例如，使用 `@Value("${DB_PASSWORD}")` 注解来注入数据库密码。

4. 运行应用程序时，环境变量的值将被注入到应用程序中，从而替换占位符，使得应用程序能够正常访问敏感信息。

使用环境变量替代明文配置信息的好处是，敏感信息不会明文存储在配置文件中，而是通过环境变量传递，从而提高了系统的安全性。此外，使用环境变量还可以方便地在不同环境中传递敏感信息，例如，在开发、测试和生产环境中，可以使用不同的环境变量传递不同的敏感信息，而无需修改配置文件。

需要注意的是，为了保护环境变量中的敏感信息，应该采取相应的安全措施，例如限制访问环境变量的权限、使用加密连接等。此外，需要确保环境变量的值不会被泄露，例如在日志中输出或通过远程调试工具传输等。

2. 加密配置文件

在 SpringBoot 中，可以使用 Jasypt (Java Simplified Encryption) 库来加密配置文件。Jasypt 是一个开源的 Java 加密库，可以为应用程序提供加密和解密功能。

以下项目可以直接从 GitCode 项目访问：<<https://gitcode.net/code-practice-course/congomall-jasypt-demo>>

以下是使用 Jasypt 加密配置文件的步骤：

1) 创建 SpringBoot 项目 congomall-jasypt-demo。

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.opengoofy.congomall.demo</groupId>
    <artifactId>congomall-jasypt-demo</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <spring-boot.version>2.3.12.RELEASE</spring-boot.version>
        <jasypt-spring-boot-starter.version>3.0.4</jasypt-spring-boot-starter.version>
```

```
<maven.compiler.source>8</maven.compiler.source>
<maven.compiler.target>8</maven.compiler.target>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

```
<dependencies>
```

```
<dependency>
```

```
<groupId>com.github.ulisesbocchio</groupId>
```

```
<artifactId>jasypt-spring-boot-starter</artifactId>
```

```
<version>${jasypt-spring-boot-starter.version}</version>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-web</artifactId>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-starter-test</artifactId>
```

```
<scope>test</scope>
```

```
</dependency>
```

```
<dependency>
```

```
<groupId>org.projectlombok</groupId>
```

```
<artifactId>lombok</artifactId>
```

```
</dependency>
```

```
</dependencies>
```

```
<dependencyManagement>
```

```
<dependencies>
```

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
```

```
<artifactId>spring-boot-dependencies</artifactId>
```

```
<version>${spring-boot.version}</version>
```

```
<type>pom</type>
```

```
        <scope>import</scope>
    </dependency>
</dependencies>
</dependencyManagement>
</project>
```

2) 开发加解密工具类。

```
import lombok.AccessLevel;
import lombok.NoArgsConstructor;
import org.jasypt.properties.PropertyValueEncryptionUtils;
import org.jasypt.util.text.BasicTextEncryptor;

@NoArgsConstructor(access = AccessLevel.PRIVATE)
public final class JasyptUtil {

    /**
     * 加密使用密钥
     */
    private static final String PRIVATE_KEY = "Eh3b59Hk";

    private static BasicTextEncryptor basicTextEncryptor = new BasicTextEncryptor();

    static {
        basicTextEncryptor.setPassword(PRIVATE_KEY);
    }

    /**
     * 明文加密
     */
    public static String encrypt(String plaintext) {
        return basicTextEncryptor.encrypt(plaintext);
    }

    /**
     * 解密
     */
    public static String decrypt(String ciphertext) {
        ciphertext = "ENC(" + ciphertext + ")";
        if (PropertyValueEncryptionUtils.isEncryptedValue(ciphertext)) {
            String plaintext = PropertyValueEncryptionUtils.decrypt(ciphertext,
```

```

        basicTextEncryptor);
        return plaintext;
    }
    return "";
}
}

```

使用单元测试，测试改类是否正常。

```

import org.junit.Test;
import org.opengooify.congomall.demo.jasypt.util.JasyptUtil;

public class JasyptUtilTest {

    @Test
    public void testEncrypt() {
        // 明文数据: chen.ma
        String encrypt = JasyptUtil.encrypt("chen.ma");
        System.out.println(encrypt);
    }

    @Test
    public void testDecrypt() {
        // 密文数据: 7Yy6LdNmwnuF6Ac7t/9W0g==
        String decrypt = JasyptUtil.decrypt("7Yy6LdNmwnuF6Ac7t/9W0g==");
        System.out.println(decrypt);
    }
}

```

3) 将 SpringBoot 中敏感配置进行加密处理。

```

server:
  port: 9283

encrypt:
  test:
    # 此处是密码的密文，要用 ENC() 进行包裹
    name: ENC(8lm2mCyFhpua3jJeJZV7pg==)

# 加密配置

```

```
jasypt:
  encryptor:
    # 指定加密密钥，生产环境需要放到启动参数
    password: Eh3b59Hk
    # 指定解密算法，需要和加密时使用的算法一致
    algorithm: PBEWithMD5AndDES
    # 指定 initialization vector 类型
    iv-generator-classname: org.jasypt.iv.NoIvGenerator
```

在应用启动类中编写代码进行测试。

```
import org.springframework.beans.factory.InitializingBean;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Application implements InitializingBean {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }

    @Value("${encrypt.test.name}")
    private String name;

    @Override
    public void afterPropertiesSet() throws Exception {
        System.out.println(name);
    }
}
```

打印结果如果是 `chen.ma` 证明加解密正确。

文末总结

环境变量替换方式有些繁琐，特别是在实际开发中需要加密的参数较多时，每一个参数都需要进行环境变量替换，这将增加开发和运维的工作量。此外，频繁调整敏感信息也将成为开发和运维的挑战。

使用 Jasypt 加密配置方式可以很好地解决这个问题。运维只需要为一个应用配置 Jasypt 单独的密钥，就可以加密所有需要加密的参数，相比之前的方式，这样做可以很好地节省时间。

但是，Jasypt 也不是那么完美，比如下述几个场景就没那么友好：

- 开发者每次需要获取新的加密字符串时，都需要联系运维，这增加了跨团队的工作量。
- 如果想查看密文字符串对应的明文，可以在本地环境启动项目并进行断点调试。但在生产环境中，只有向运维要求密钥后才能进行查看。
- 对于审计比较严格的项目，会要求定期更换项目的密钥。

我们对 Jasypt 进行了单独的封装，独立出了一个加密中台项目。在项目启动时，应用会向加密中台发送请求，然后将 ENC 中的字符串替换为加密中台自定义的字符串。

值得一提的是，经过改造后 ENC 中的字符串不再是密文，而是加密中台自定义的字符串，这个字符串可以用来匹配对应的明文数据。

更新: 2023-07-17 18:40:20

原文: <<https://www.yuque.com/magestack/12306/xpnpa9cg3v620nog>>